March 12, 2026

Design of a Lexical Analyzer Generator

- We assume that we have a specification of a lexical analyzer of the form:

$$\begin{bmatrix} P_1 & \{\text{action 1}\} \\ P_2 & \{\text{action 2}\} \\ \cdots & \\ P_n & \{\text{action n}\} \end{bmatrix}$$

  where each pattern P_i is a regular expression and each action action i is the procedure to be executed whenever a lexeme matched by P_i is found in the input.

- Our problem is to construct a recognizer that looks for lexemes in the input buffer. If more than one pattern matches, the recognizer is to choose the longest lexeme matched. If there are more than one lexeme of the longest length, then the first listed matching pattern is chosen.

- Example: Let us consider the following program consisting of the regular expressions (actions are omitted):

$$\begin{bmatrix} a & \{\} \\ abb & \{\} \\ a^*b^+ & \{\} \end{bmatrix}$$

  we can convert the DFA's into one combined NFA as shown below:

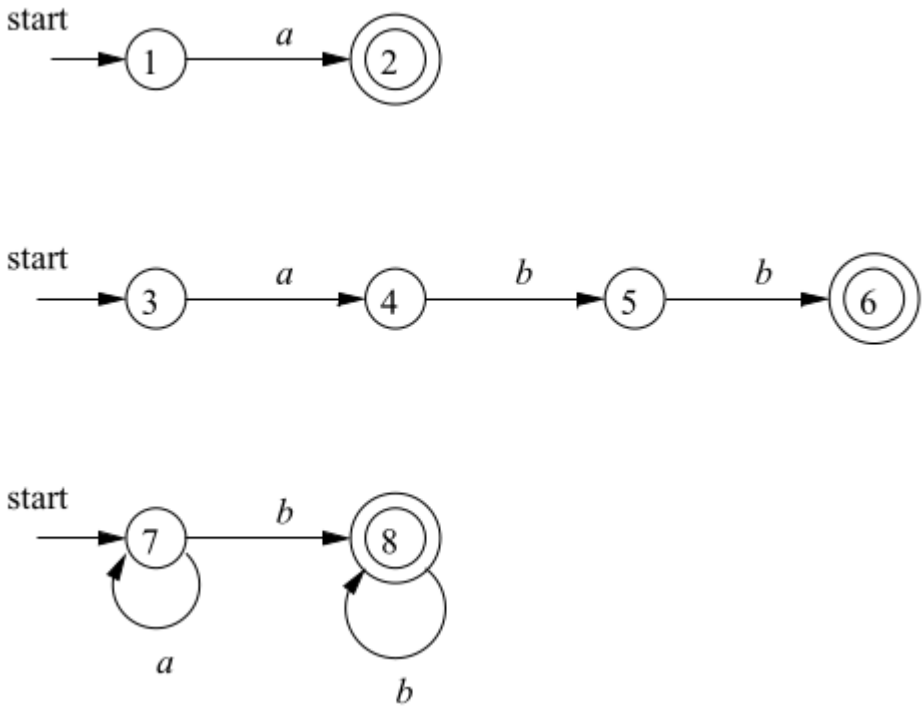


*Figure 1: DFAs corresponding to a, abb and a*b+*

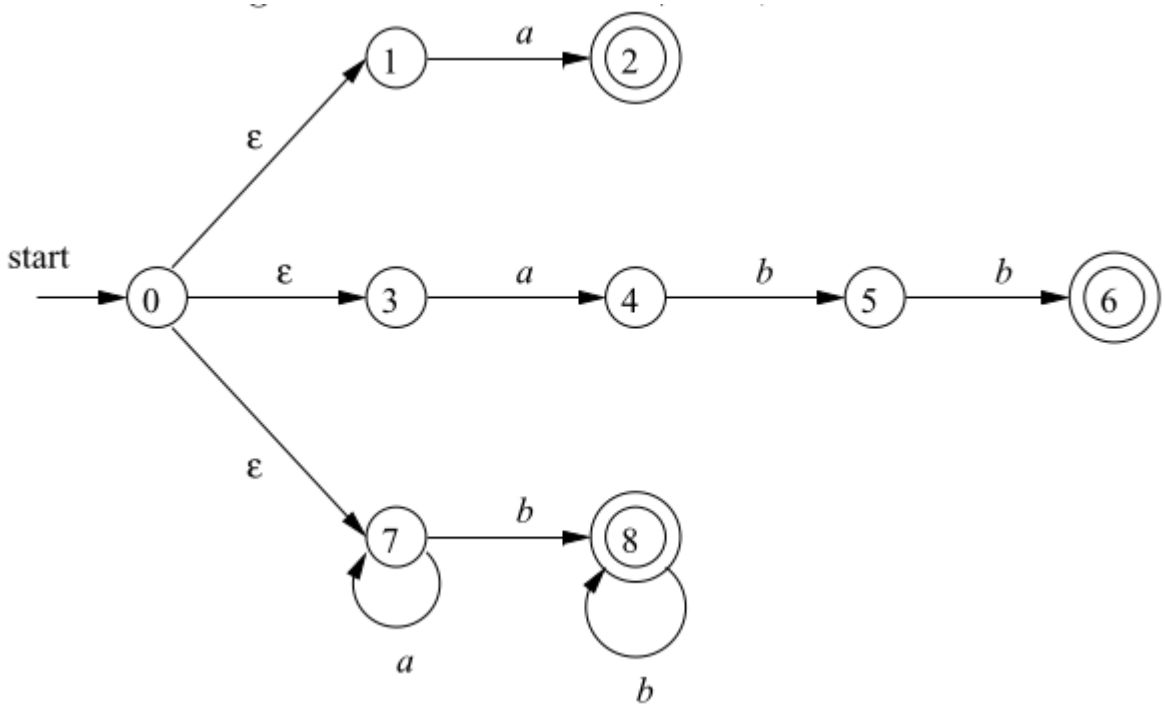


Figure 2: Combined NFA

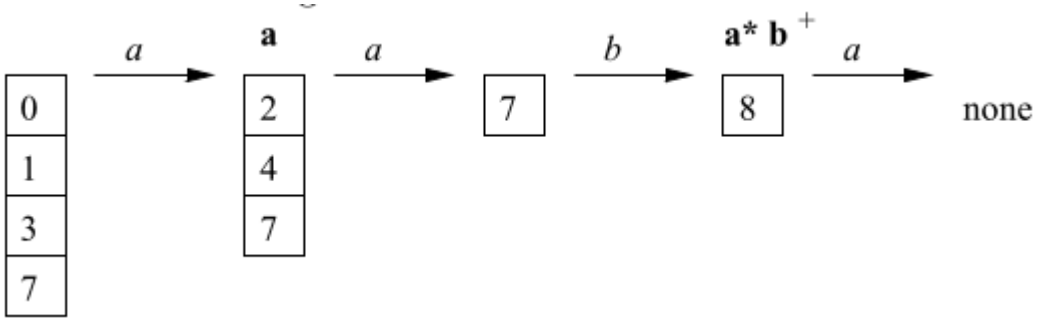


Figure 3: States during input

At state 8, no transition is possible, so we have reached termination. Since the last match occurred after reading the third character, we report that the third pattern a^*b^+ has matched lexeme aab.

Important State

- An important state is that state of an NFA which has a non-ϵ out transition.
- In the subset construction algorithm, only the important states in T are used during the determination of the ϵ-closure(move(T, a)), the set of states that is reachable from T for input a.

Augmented Regular Expression

- The resulting NFA obtained from a regular expression has exactly one accepting state, but the accepting state is not important because it has no transition leaving it.
- By concatenating a unique marker $\#$ to the regular expression r, we give the accepting state of r on transition r making it an important state of NFA for $(r)\#$.

Representation

- Consider the regular expression $(a|b)^*abb\#$, we represent this by syntax tree with basic symbols at the leaves and operators in the interior nodes.

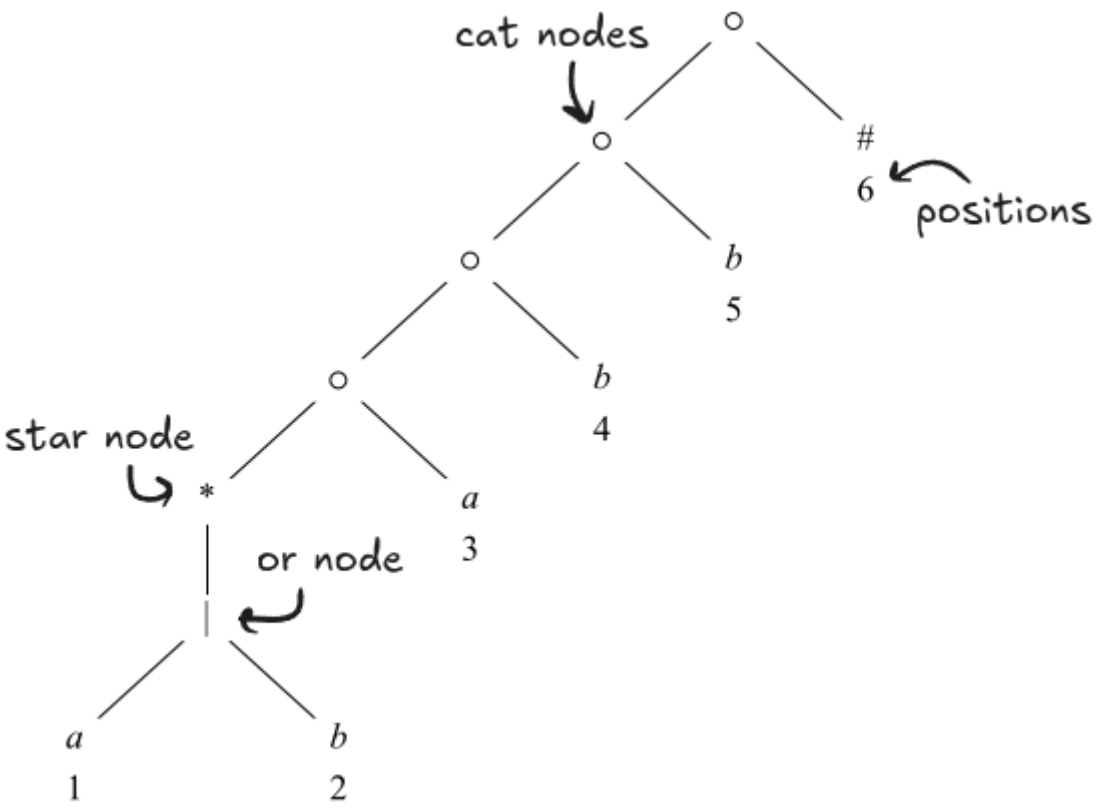
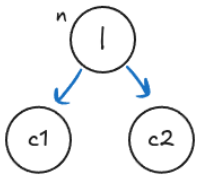
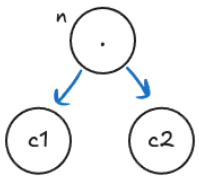
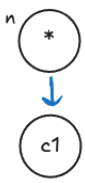


*Figure 4: Syntax tree of (a|b)*abb#*

Functions Computed From Syntax Tree

1. **FirstPos:** At each node n of a syntax tree of a regular expression, we define a function $firstPos(n)$ that gives the set of positions that can match the first symbol of a string generated by the sub-expression rooted at n.
2. **LastPos:** We define a function $lastPos(n)$ that gives the set of positions that can match the last symbol in such a string.
   E.g. in the regular expression $(a|b)^*abb\#$, $firstPos(root) = \{1, 2, 3\}$ and $lastPos(root) = 6$
3. **Nullable:** The nodes that are the roots of sub-expressions that generates languages that include the empty string are called nullable. We define $nullable(n)$ to be true if node n is nullable else false.
4. **FollowPos:** If i is a position, then $followPos(i)$ is the set of positions j such that there is some input string $\ldots cd \ldots$ such that i corresponds to this occurrence of c and j to this occurrence of d.
   E.g. in the regular expression $(a|b)^*abb\#$, $followPos(1) = \{1, 2, 3\}$

Rules for Computing

Node n	$nullable(n)$	$firstPos(n)$	$lastPos(n)$
n is a leaf labeled ϵ	true	ϕ	ϕ
n is a leaf labeled with position i	false	$\{i\}$	$\{i\}$
	$nullable(c_1)$ or $nullable(c_2)$	$firstPos(c_1) \cup firstPos(c_2)$	$lastPos(c_1) \cup lastPos(c_2)$
	$nullable(c_2)$ $nullable(c_1)$ and	If $nullable(c_1)$ then $firstPos(c_1) \cup firstPos(c_2)$ Else $firstPos(c_1)$	If $nullable(c_1)$ then $lastPos(c_1) \cup lastPos(c_2)$ Else $lastPos(c_2)$
	true	$firstPos(c_1)$	$lastPos(c_1)$

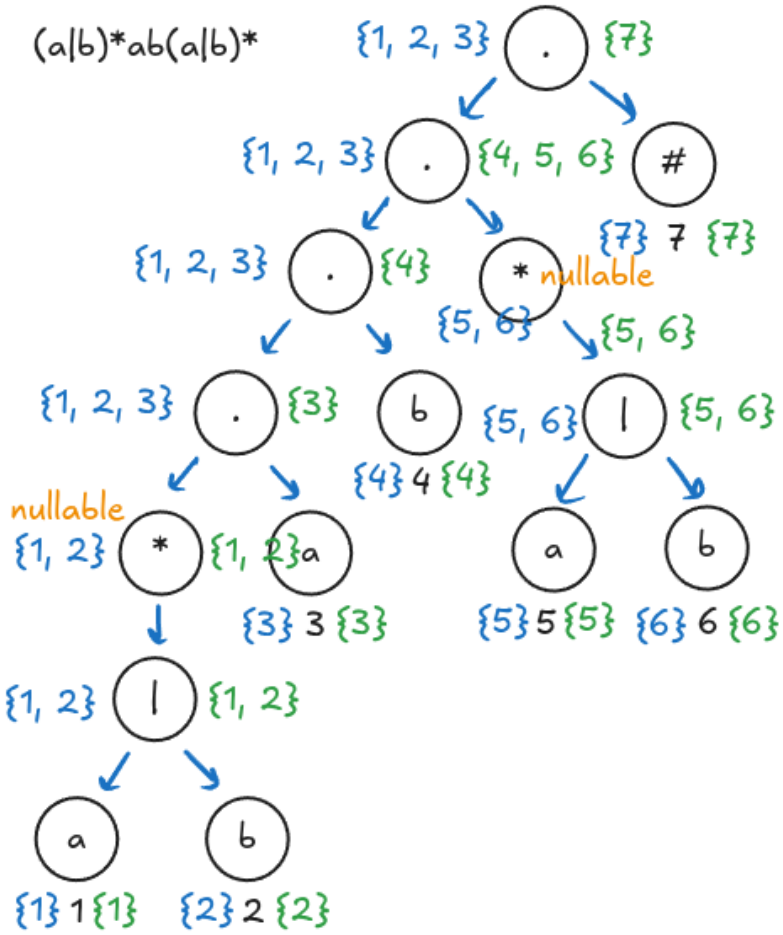


Figure 5: FirstPos, LastPos and Nullable

Rules for computing FollowPos

The function $followPos(i)$ tells us which positions can follow position i in the syntax tree.

1. If n is a cat-node with left child c_1 and right child c_2, then for every position i in $lastPos(c_1)$, all positions in f$irstPos(c_2)$ are in $followPos(i)$.
2. If n is a star-node, and i is a position in $lastPos(n)$, then all positions in $firstPos(n)$ are in $followPos(i)$.

n	$followPos(n)$
1	{1, 2, 3}
2	{1, 2, 3}
3	{4}
4	{5, 6, 7}
5	{5, 6, 7}
6	{5, 6, 7}
7	ϕ